

Bridges and Articulation Points

Lecture 7

CSEN 1038 Advanced Data Structures and Algorithms

Table of contents

1. Graphs Fundamentals Recap
2. Single Points of Failure
3. The DFS Tree
4. Tarjan's Algorithm

Graphs Fundamentals Recap

Graph Terminology & Representation

- **Graph** $G = (V, E)$: A collection of vertices (V) and edges (E).
- **Connected Component**: A subgraph where any two vertices are connected by a path.
- **Adjacency List**: The standard representation for efficient graph traversal.

```
vector<int> adj[N];
```



Watch animated illustration

Depth-First Search (DFS) Recap

- Traverses as deeply as possible along each branch before backtracking.
- Uses a recursion stack implicitly (or an explicit stack).
- **Time Complexity:** $\mathcal{O}(V + E)$



Watch animated DFS recap

Single Points of Failure

What is a Bridge?

Definition: Bridge (Cut Edge)

An edge in an undirected graph whose removal strictly **increases the number of connected components**.

- Represents a critical bottleneck in infrastructure (e.g., a single fiber-optic cable linking two data centers).



Watch animated bridge example

What is an Articulation Point?

Definition: Articulation Point (Cut Vertex)

A vertex whose removal (along with all its incident edges) strictly **increases the number of connected components**.

- Represents a critical node (e.g., a master router or central server).



Watch animated articulation-point example

The DFS Tree

When performing a DFS on an undirected graph, all edges are classified into exactly two types:

1. **Tree Edges:** Edges traversed by DFS to discover new, unvisited nodes.
2. **Back Edges:** Edges connecting a node to one of its ancestors in the DFS tree.



Watch animated DFS-tree illustration

Tracking the DFS State

To find bridges/APs efficiently without running DFS multiple times, we track two values for every node:

- `depth[u]`: The discovery depth. The exact level the DFS reached node u in the recursion tree.
- `low[u]`: The lowest depth reachable from u (or its descendants) using **at most one back edge**.

Tarjan's Algorithm

The Condition: An edge (v, u) (where u is a child of v in the DFS tree) is a bridge if and only if:

$$\text{low}[u] > \text{depth}[v]$$

Intuition: Node u (and its descendants) has absolutely no back edge reaching v or any ancestor of v . Removing (v, u) cuts them off completely.

Tarjan's Algorithm for Bridges

Time Complexity: $\mathcal{O}(V + E)$

```
const int inf = 1e9;
vector<int> vis(n), low(n, inf), depth(n), is_bridge(m);

void dfs_bridges(int v, int p) {
    vis[v] = 1;
    for (const auto& [u, edge_id] : g[v]) {
        if (u == p) continue;
        if (vis[u]) {
            low[v] = min(low[v], depth[u]);
            continue;
        }
        depth[u] = 1 + depth[v];
        dfs_bridges(u, v);
        low[v] = min(low[v], low[u]);
        if (low[u] > depth[v]) is_bridge[edge_id] = 1;
    }
}
```

The Condition: A node v is an articulation point if it has a child u such that:

$$\text{low}[u] \geq \text{depth}[v]$$

Intuition: If v is removed, child u has no strict back edge to an ancestor of v to escape.

The Root Exception

If v is the root of the DFS tree, it has no ancestors! Therefore, the root is an AP if and only if it has **strictly more than 1 child** in the DFS tree.

Tarjan's Algorithm for Articulation Points

Time Complexity: $\mathcal{O}(V + E)$

```
const int inf = 1e9;
vector<int> vis(n), low(n, inf), depth(n), cut(n);
void dfs_ap(int v, int p) {
    vis[v] = 1;
    int kids = 0;
    for (int u : g[v]) {
        if (u == p) continue;
        if (vis[u]) {
            low[v] = min(low[v], depth[u]);
            continue;
        }
        kids++;
        depth[u] = 1 + depth[v];
        dfs_ap(u, v);
        low[v] = min(low[v], low[u]);
        if (low[u] >= depth[v]) cut[v] = (p != -1 || kids > 1);
    }
}
```


Questions?

- <https://youtu.be/jDi0X7KvZnE>
- <https://codeforces.com/blog/entry/68138>
- <https://cp-algorithms.com/graph/bridge-searching.html>
- <https://cp-algorithms.com/graph/cutpoints.html>